

Task Manager Assessment

By Tyler Tang, 4/4/2026

Project Documentation - Design decisions and solutions

1. Starting from Scratch

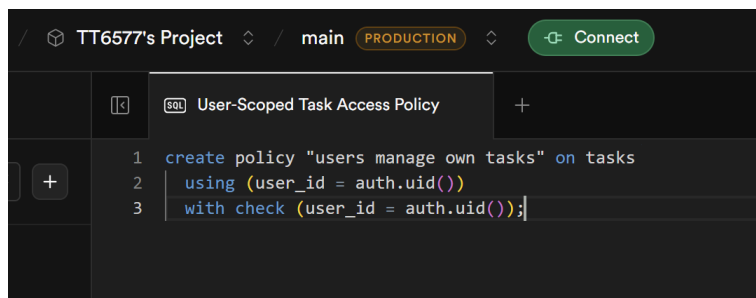
This mini-project was really really tough for me. Before I started the project, I had no idea how to use Node.js or Supabase, code in TypeScript or React, or any of the libraries. I know that you're only supposed to spend 3-6 hours, 12 hours max, on this project, but this week I spent about 20 hours, 5 hours across 4 days, just constantly working on this project. If I'm going to be honest, I used a lot of AI in this project. I used Claude to explain a ton of stuff, including downloading and setting everything up, writing a lot of code, and explaining how React and TypeScript works. It helped that I have a lot of previous experience with JavaScript and HTML, but man it was super tough still.

I went back and reread all the code I didn't write because I think it's important for programmers to understand the code that is written, even if AI does a lot of work, because ultimately if AI screws up or one day goes out of fashion I'll need to fix everything myself. I think AI should purely enhance my work, but not replace anything, although I did need a lot of help getting on my feet for this project.

2. Intro to Vite, Supabase, TypeScript, React

Just to start, I had Claude help me set up my little Vite server, Supabase database, and introduce me to TypeScript. I still haven't learned async in depth in class yet, and I think it would've taken too long to learn by myself, so I just let Claude generate code on loading stuff from databases. Previously in my other website project, I stored things on a csv on the website itself, and just added stuff to a google sheets using their api, so I didn't understand how to use a real database, but now I have a basic understanding.

I did a little manual coding setting up the columns with a lot of help. I used GeeksForGeeks a bunch to learn the basics of React and TypeScript. I first set up the task entry form, and it was kind of rough trying to set up the form, but the more I did, the more comfortable I felt, and by the end of making it, I felt I had a working knowledge of everything.

A screenshot of a Supabase SQL editor interface. The top bar shows the project name 'TT6577's Project', the current branch 'main', and a 'PRODUCTION' environment tag. A 'Connect' button is visible on the right. The main editor area displays a SQL query for creating a policy. The query is as follows:

```
1 create policy "users manage own tasks" on tasks
2 using (user_id = auth.uid())
3 with check (user_id = auth.uid());
```

Here's some code Claude wrote for me. I don't know how to write SQL, I only know how to access SQL databases, so I needed help making something like this. Creating the table was simple enough, though, I just added the

variables outlined in the assignment document

3. Picking up pace, setting goals, getting comfortable

After getting through the rough start with lots of help from Claude, I created a little list of things I wanted to implement in the final version. I knew I wasn't going to be able to implement everything given my experience and knowledge handicap, so I decided to just choose the two/three of the easier advanced features to add. I decided to add features 4-7: labels, a board summary, search/filtering, and a due date/urgency tracker. I also added the advanced table bonus fields, because that isn't too hard to implement either.

I just started by implementing the most basic functions, without caring too much about the design. I just created white columns with black text, and set up a basic sorting system for tasks. As I worked, I started to decide more on the design of the page. I used a color scheme similar to Linear, like a dark-mode purple modern style. I like how clean it looks and it isn't too complex. I added a color tint to the task objects based on priority, so it is easier to tell at a glance what is urgent and what isn't.

I had to do a whole lot of formatting things because it's a real headache how having so many variable length objects on a page can cause things to break. A consistent problem was that having the title of a task being too long or having a single word be too long could break formatting by going off the page or by stretching the status column. I added a truncate function and added some css styling for breaking up and wrapping words to fit long descriptions in the details panel or the task objects.

4. Adding libraries for complex features

I don't have extensive knowledge of libraries for web development, so I asked Claude to suggest libraries that I could use. The libraries I ended up using are dnd kit for drag and drop interfaces, Lucide for React for cute little icons, and Motion for React (framer-motion). Dnd kit is super complex so I just copied a bunch of code online and used Claude to debug a whole host of issues that came along. Motion is also really crazy and I was tired of debugging so much, so I ended up just scrapping my own attempts at implementation and used Claude's code, which worked surprisingly well. Lucide is very straightforward, so I didn't need much help, and added them where appropriate to enhance the visuals.

5. Last stretch, debugging, polishing visuals, running tests

After most of the final features were working, it was onto polishing. Most of my Motion implementation was done in this phase, and it's when I decide to choose an actual color palette instead of choosing random colors on vibes. I fixed a bunch of the formatting, including removing the stupid empty panels on the sides. It literally took me the entire project to realize that the default root div that React creates has a margin that can't be simply removed by setting all divs to have no margin. I redid some of my flex code to be a set number of pixels wide, and

added some resizing css tricks to make sure nothing moves weirdly when large task objects are added or moved around or clicked on.

I added the project to GitHub, added a bunch of changes, added it onto Vercel, added a bunch of changes, and here we are.

6. Final thoughts

I actually really enjoyed this project. I thought it was interesting to learn to code in a semi-professional environment, and learning a new language is always really fun and makes me feel smart. I'm quite proud of myself for picking things up so quickly, and it's nice that I have something to keep my brain moving during spring break. I think I'm fairly proficient at Node, React and TypeScript now, and I feel more caught up now that I have tried a bit of "real" programming. Now I have another cool thing to put on my portfolio.

Vercel Link	https://kanban-app-prototype.vercel.app/
GitHub Repo	https://github.com/TT6577/kanban-app-prototype (it says there's another contributor, but that is just my school github account that i forgot to switch out of)

Database Schema (SQL)
<pre>create table tasks (id uuid primary key default gen_random_uuid(), title text not null, status text default 'todo', priority text default 'normal', description text, date date, labels text[] default '{}', user_id uuid references auth.users(id), created_at timestamp default now()); alter table tasks enable row level security; create policy "users manage own tasks" on tasks</pre>

```
using (user_id = auth.uid())  
with check (user_id = auth.uid());
```

Setup Instructions:

Prerequisites: Node.js installed

1. Clone the repository:

```
git clone https://github.com/TT6577/kanban-app-prototype.git
```

2. Install dependencies:

```
cd kanban-app  
npm install
```

3. Create a .env file in the root directory with the following:

```
VITE_SUPABASE_URL= supabase url (using supabase database id)  
VITE_SUPABASE_ANON_KEY= supabase anon key
```

4. Start the development server:

```
npm run dev
```

5. Open <http://localhost:5173> in your browser

Advanced Features and how they work

- **Labels/Tags**

You can add labels/tags when you create a task. You can click on a tag to filter all tasks with the selected tag, and click on it again to remove the filter. All tags with the same label have the same color, and different labels have different colors. You can add multiple tags by separating them with a space.

- **Due Date Indicators**

You can add a due date when you create a task. If the due date is within 2 days, then the task object will have a red outline and a little red icon showing that it is urgent. However, if it is in the “Done” column, it will not have the red outline or icon, and it will not count as an urgent task.

- **Search and Filtering**

There is a search bar above the board summary, and tasks will be dynamically filtered as you type into it. You can filter by tag by clicking on the tag, and un-filter by clicking on the tag again. There are two small text boxes that appear at the bottom below the columns when you search or

filter by tag, showing you what you're searching for and what tag you're filtering by, just in case it is not obvious, since the search bar is sometimes not visible and it is not always clear when you're filtering by a tag.

- **Board Summary/Stats**

When no task is selected, there is a board summary that shows a bunch of useful data. It shows the number of total tasks, urgent tasks, completed tasks, and tasks left to complete. It updates dynamically and automatically when objects are moved, including when urgent tasks are moved in and out of the "Done" column.

What I'd improve with more time

I think I could improve the website by a good amount with more time. I'm confident that if I was given a full work week I could complete the whole project with all the advanced features and it would be very polished. I spent a lot of my time just learning about all these new tools and just fixing bugs, so if I were to retry this assignment again I think I could get a lot farther in the same amount of time.

I think for starters I'd improve my filter and the ability to edit tasks after their creation. At the moment you can't edit any tasks, you must delete it and remake it to what you want. So that means if you create a new label you can't retroactively add it to other tasks, and once all tasks with a certain label get deleted the label is also deleted. Labels aren't saved so you have to manually type in every label you want. Additionally, you can only filter by one label at a time, which isn't as good as having multiple labels.

I think the "time constraint" I placed on myself limited a lot of stuff I could do on this project. I really did try hard to learn everything and make as polished of a website as I could, but ultimately I am held back by my inexperience. I think I optimized my learning and coding a lot with careful planning and help from AI, but I have a long ways to go. Still, I think I show promise in my ability to adapt to new environments and learn super fast, as well as a coding style that meshes well with AI, which I've heard is prevalent in tech jobs nowadays.

I hope you enjoy using my website!